UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA



Logo UMG

ANÁLISIS DE SISTEMAS II

Ingeniero Richard Ortiz

Tarea 4: Arquitectura Repository

Módulo 29: QA funcional, pruebas manuales y matriz de regresión

Didhyer Alexander Ortíz Guevara

Carnet: 1890-23-8474

13 de septiembre de 2026

Índice

1. Introducción
2. Problema
3. Objetivos
4. Asignación y alcance del módulo
5. Requisitos y criterios principales
6. Arquitectura y federación
7. Distinción del patrón Repository
8. Implementación por capas
9. Modelo PostgreSQL
10. API entregada
11. Seguridad y privacidad
12. Pruebas y evidencia
13. Matriz, defecto y re-prueba
14. Decisiones, riesgos y limitaciones
15. Conclusiones
16. Bibliografía

Informe técnico: Repository aplicado al Módulo 29 de QA funcional y regresión

Dato ejecutivo	Valor
Estudiante	Didhyer Alexander Ortiz Guevara
Carnet	`1890-23-8474`
GitHub	`DidhyerOrtiz`
Asignación	Tarea 4, variante `mod29`, antecedente histórico issue `#28`
Rama	`feat/mod29-qa-regresion`
Base	`origin/develop` en `0050f4f`
Tip de código y pruebas verificado	`80f8f37`
Pull request	https://github.com/compilations-teams/sistema-hospitalario-integrado-SistenasII-2026/pull/63
Commit evaluado de código y evidencia	`bef242c`
Corte final reforzado	Worktree verificado sobre `f52d969`, con diagramas e informe incorporados

1. Introducción

El presente informe describe la implementación del Módulo 29, orientado al registro de QA funcional, pruebas manuales y matriz de regresión dentro del Sistema Hospitalario Integrado. Su propósito no es ejecutar automáticamente el sistema bajo prueba, sino ofrecer un vertical de backend capaz de conservar qué requisito se evaluó, cuál fue el criterio de aceptación, qué resultado observó el analista, qué evidencia declaró, qué defecto surgió y cómo terminó su re-prueba. Esta trazabilidad convierte una afirmación informal de calidad en registros consultables y delimitados por hospital.

La entrega parte del antecedente histórico issue `#28` y se desarrolló en `feat/mod29-qa-regresion`. El corte técnico examinado comprende diez commits sustantivos sobre `origin/develop`, desde la especificación y modelado hasta pruebas HTTP y PostgreSQL. El código verificado termina en `80f8f37`; los documentos de evidencia son posteriores y no cambian ese punto de referencia.

El concepto central de la tarea es Repository. En este trabajo se distingue cuidadosamente el repositorio Git, que conserva archivos e historial, del patrón Repository, que abstrae la persistencia del dominio. También se lo separa de la arquitectura federada CENTRAL/HOSPITAL, responsable de definir propiedad y disponibilidad de datos. Las tres ideas se relacionan, pero no son intercambiables.

2. Problema

Una versión del sistema hospitalario puede incorporar cambios en admisión, laboratorio u otras capacidades, pero una lista de commits no demuestra por sí sola que los requisitos continúen funcionando. Sin una matriz de regresión se pierde la relación entre requisito, criterio de aceptación, caso, versión objetivo, ejecución y defecto. Además, una corrección puede declararse terminada sin conservar la ejecución que mostró el fallo ni una re-prueba que confirme el resultado.

El contexto hospitalario añade restricciones relevantes. Los registros de QA pertenecen al hospital que ejecuta la verificación, incluso cuando el caso apunta a una capacidad CENTRAL. Una caída de CENTRAL no debe impedir que el analista registre localmente lo observado. A la vez, una evidencia puede contener información sensible, por lo que el backend debe exigir referencias privadas y metadatos redactados, rechazar claves peligrosas y no aceptar el hospital propietario desde el payload del cliente.

La solución debía ser mínima y honesta. No se requería una interfaz gráfica, automatización E2E, un almacén físico de artefactos ni sincronización distribuida. Tampoco debía representar como implementadas capacidades como versionado de casos, snapshots, outbox o idempotencia. El reto consistió en entregar un flujo vertical completo y verificable sin ampliar artificialmente el alcance.

3. Objetivos

3.1 Objetivo general

Implementar un módulo de QA funcional y regresión que registre casos trazables, campañas, ejecuciones, evidencias declaradas, defectos y re-pruebas, aplicando el patrón Repository y persistencia PostgreSQL aislada por hospital.

3.2 Objetivos específicos

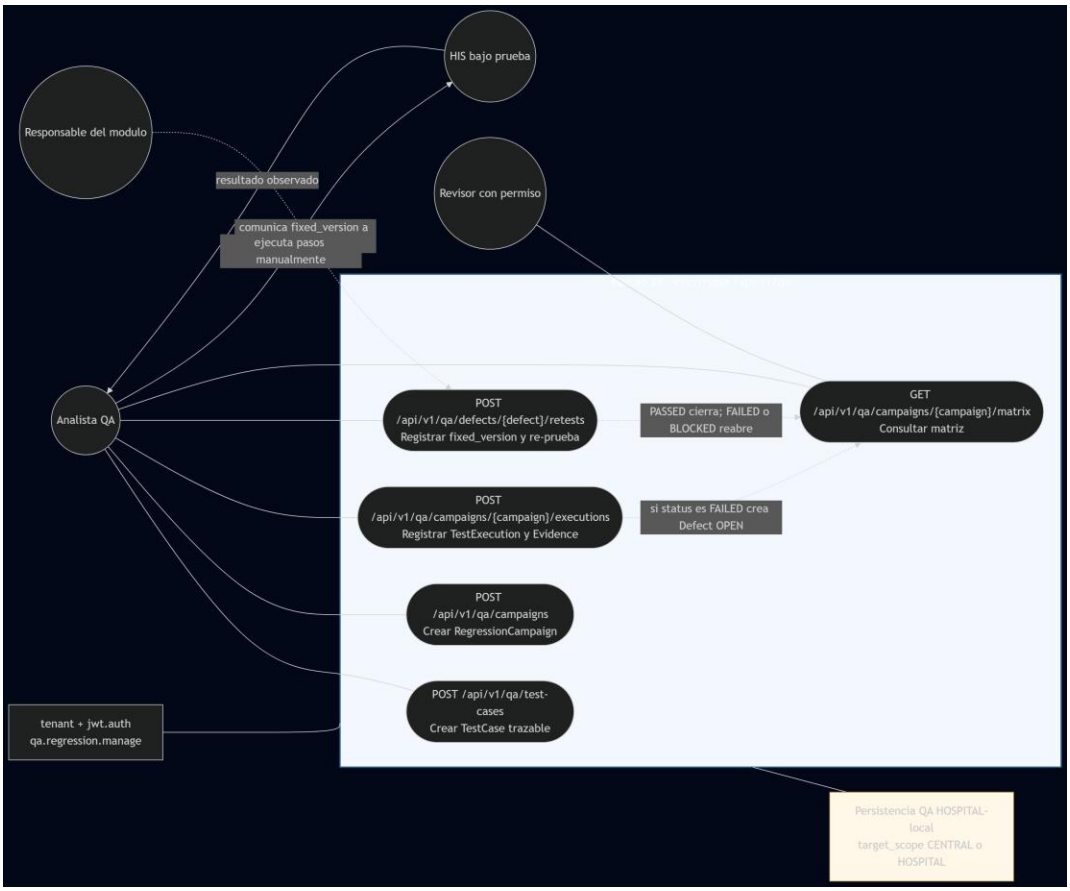
- Vincular cada caso con un requisito y un criterio de aceptación obligatorios.
- Agrupar casos del mismo hospital en una campaña asociada a una versión objetivo.
- Registrar resultados `PASSED`, `FAILED` o `BLOCKED` con evidencia saneada.
- Crear un defecto `OPEN` en la misma transacción de una ejecución inicial fallida.
- Registrar una nueva ejecución de re-prueba y cerrar o reabrir el defecto según el resultado.
- Exponer una matriz que consolide casos, último estado, ejecuciones y defectos.
- Proteger todas las operaciones mediante JWT, tenant y permiso específico.
- Validar el repositorio Eloquent y las restricciones relacionales en PostgreSQL 17 real.
- Mantener trazabilidad documental de un defecto de implementación y de su corrección.

4. Asignación y alcance del módulo

La variante `mod29` corresponde a “QA funcional, pruebas manuales y matriz de regresión”. El actor principal es el Analista QA con permiso `qa.regression.manage`. El responsable del módulo probado realiza correcciones fuera de este vertical y comunica la versión que el analista registra como `fixed_version`. Un revisor puede consultar la matriz si posee el mismo permiso; no se implementó un permiso de solo lectura.

El alcance incluye cinco operaciones: crear caso, crear campaña, registrar ejecución, re-probar defecto y consultar matriz. El caso posee alcance objetivo `CENTRAL` o `HOSPITAL`, referencias lógicas, título, precondiciones opcionales, pasos y resultado esperado. La campaña conserva nombre, versión objetivo y casos. Cada ejecución registra estado, resultado real, ejecutor y una evidencia. Un fallo inicial incorpora pasos de reproducción, severidad y versión detectada. La re-prueba exige una versión corregida y conserva un enlace a la ejecución original.

Quedan fuera editar o eliminar casos y campañas, administrar estados de campaña, subir archivos, comprobar físicamente evidencia, consultar requisitos remotos, corregir el sistema evaluado y automatizar pruebas de interfaz. No hay versionado ni snapshot del caso, outbox, idempotencia o sincronización CENTRAL/HOSPITAL. Estas exclusiones son decisiones de alcance verificables en el código, no funcionalidades pendientes presentadas como terminadas.



Casos de uso del Módulo 29

Figura 1. Casos de uso expuestos por las cinco rutas. Fuente editable: `casos-de-uso.mmd`.

5. Requisitos y criterios principales

La trazabilidad mínima exige `requirement_ref` y `acceptance_criterion_ref`. El alcance solo admite `CENTRAL` o `HOSPITAL`; todos los casos requieren título, pasos no vacíos y resultado esperado. Una campaña necesita nombre, versión y al menos un caso, sin repetidos, y solo puede asociar registros visibles en el mismo `hospital_id`.

Una ejecución debe corresponder a un caso asociado a la campaña. Los estados válidos son `PASSED`, `FAILED` y `BLOCKED`. Todos exigen resultado observado y evidencia. Si la ejecución inicial falla, también son obligatorios los pasos de reproducción, una severidad `LOW`, `MEDIUM`, `HIGH` o `CRITICAL`, y `detected_version`. El resultado debe crear ejecución, evidencia y defecto `OPEN` de forma atómica. Una ejecución aprobada o bloqueada no crea defecto inicial.

La re-prueba exige `fixed_version`, nueva evidencia y resultado. No sobrescribe la ejecución inicial: crea otra ejecución con `retest_of_execution_id` y `defect_id`. Si pasa, el defecto queda `CLOSED` y registra `closed_at`; si falla o queda bloqueada, queda `REOPENED` sin fecha de cierre. Un defecto cerrado rechaza nuevas re-pruebas.

La evidencia debe declarar una ruta bajo `private/qa/`, un checksum con sintaxis SHA-256 y metadatos con `redacted=true`, `mime` y `size`. La validación recursiva rechaza las claves `password`, `token`, `authorization`, `cookie` y `secret` sin distinguir mayúsculas. Esta regla reduce el riesgo de registrar secretos, aunque no reemplaza una revisión humana de datos personales.

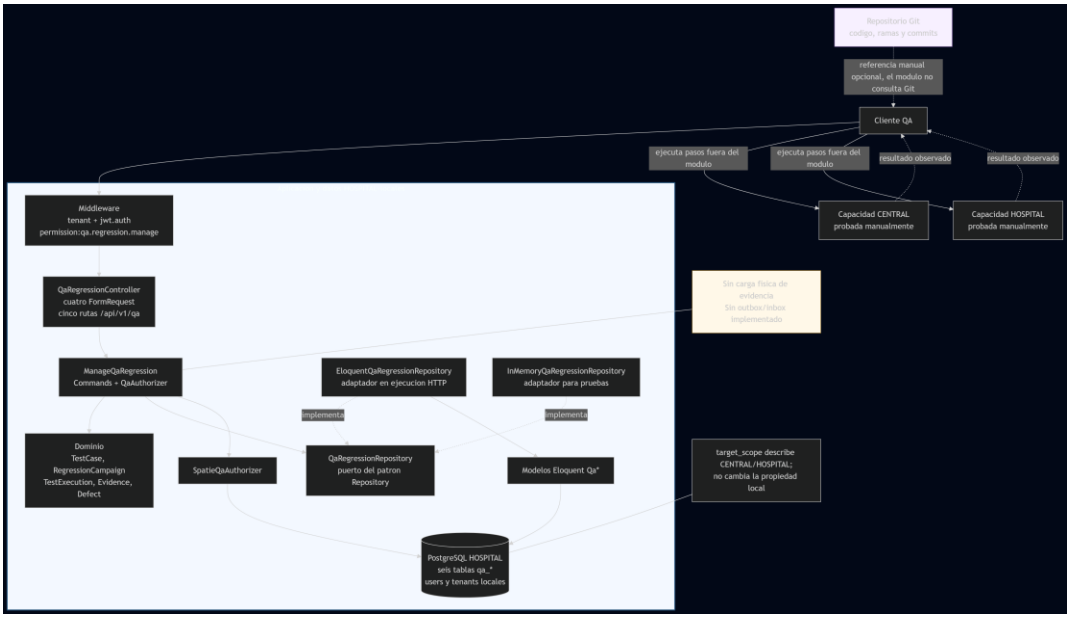
6. Arquitectura y federación

La solución sigue un flujo por capas. HTTP recibe y valida solicitudes; Application orquesta el caso de uso; Domain mantiene entidades e invariantes; Infrastructure implementa autorización y persistencia. El diagrama de componentes representa estos límites y distingue las capacidades observadas manualmente de las dependencias reales del módulo.

`QaRegressionController` convierte los cuatro `FormRequest` en comandos y traduce excepciones conocidas a respuestas HTTP. `ManageQaRegression` autoriza, crea entidades y solicita operaciones significativas al Repository. El dominio contiene `TestCase`, `RegressionCampaign`, `TestExecution`, `Evidence` y `Defect`, junto con enums y excepciones. Infraestructura incorpora autorización Spatie, modelos Eloquent y dos implementaciones del puerto.

La federación establece que los registros QA son locales al nodo HOSPITAL. `target_scope=CENTRAL` solo expresa qué capacidad se evaluó; no traslada la propiedad del registro ni abre una transacción remota. Las referencias de requisito, criterio y versión son cadenas lógicas. No existen JOINs ni FKs hacia CENTRAL. Por ello, el analista puede registrar que una función CENTRAL estaba indisponible sin depender de CENTRAL para guardar ese resultado.

Las FKs a `tenants` y `users` son locales y compartidas en la misma base. El aislamiento se aplica en varias capas: middleware resuelve el tenant; la autorización comprueba usuario y hospital; el caso de uso busca entidades dentro de ese hospital; el Repository filtra consultas por `hospital_id`; y las claves compuestas refuerzan relaciones de pertenencia en PostgreSQL.



Vista de componentes y límites

Figura 2. Componentes implementados y límites HOSPITAL/CENTRAL. Fuente editable: `componentes.mmd`.

7. Distinción del patrón Repository

El repositorio Git contiene la rama, los diez commits y el historial desde `0050f4f`. No participa en tiempo de ejecución. Aunque `target_version`, `detected_version` y `fixed_version` puedan recibir nombres semejantes a una etiqueta o commit, el módulo no consulta Git ni valida que esas cadenas existan.

El patrón Repository sí forma parte del software. `QaRegressionRepository` define nueve operaciones orientadas al dominio: guardar y buscar caso, guardar y buscar campaña, registrar ejecución, buscar ejecución, buscar defecto, registrar re-prueba y construir la matriz. No es un CRUD genérico. Métodos como `recordExecution` y `recordRetest` expresan límites transaccionales y reglas que serían difíciles de representar mediante simples altas o actualizaciones independientes.

`InMemoryQaRegressionRepository` permite probar `ManageQaRegression` sin infraestructura externa. `EloquentQaRegressionRepository` adapta esas operaciones a modelos y transacciones reales. `AppServiceProvider` enlaza el puerto con Eloquent para la ejecución HTTP. Esta inversión de dependencia evita que el caso de uso conozca tablas, consultas o detalles de Laravel, y facilita comparar el comportamiento rápido en memoria con restricciones efectivas de PostgreSQL.

La arquitectura federada, por su parte, responde dónde pertenece el dato y de qué nodo depende. No sustituye al patrón Repository. Esta separación conceptual es un resultado

importante de la tarea: Git conserva el producto, Repository abstrae persistencia y la federación delimita propiedad y disponibilidad.

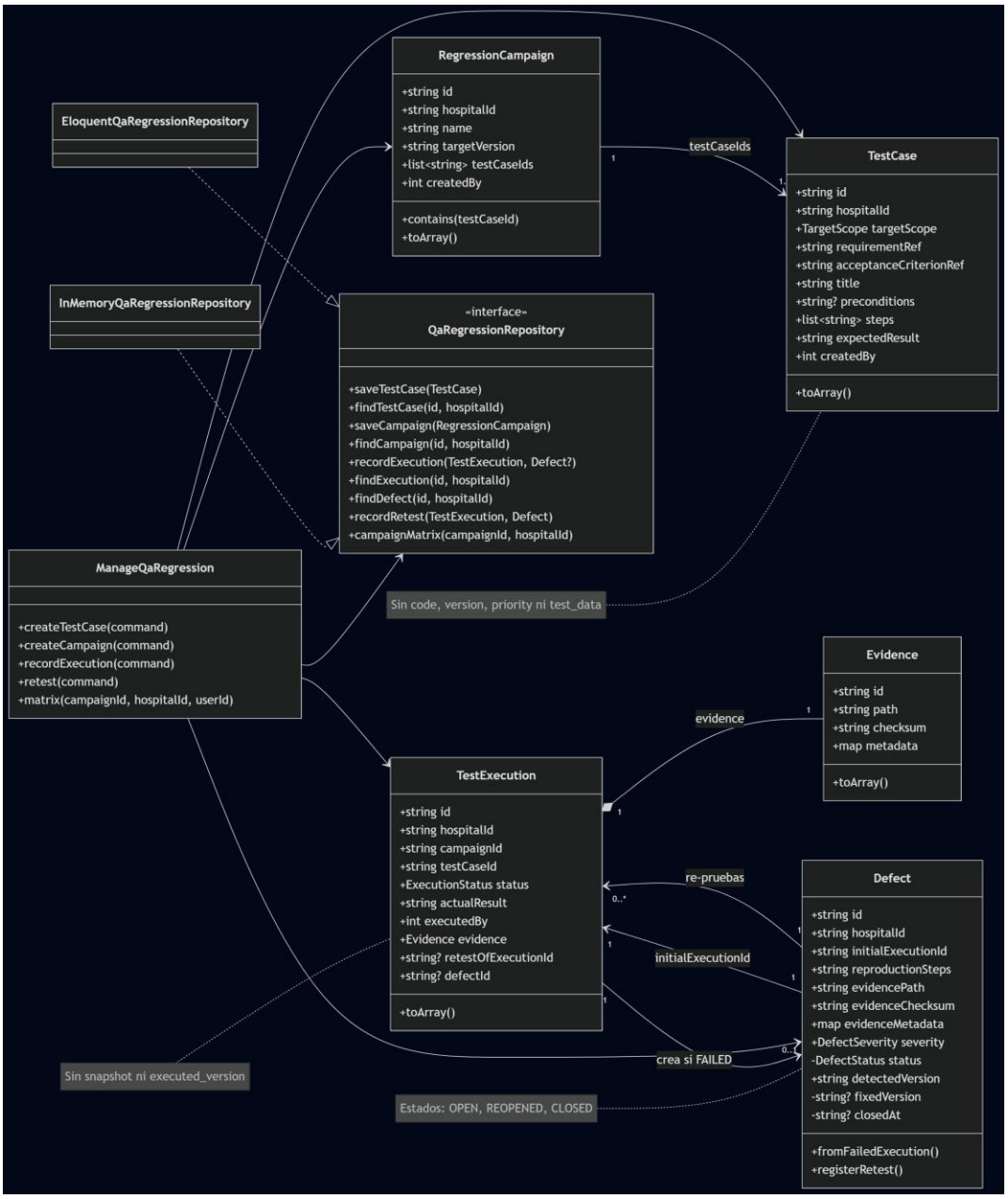


Diagrama de clases y Repository

Figura 3. Entidades, caso de uso, puerto y adaptadores. Fuente editable: `diagrama-clases.mmd`.

8. Implementación por capas

8.1 Dominio

Las entidades validan contenido mínimo y estados permitidos.
`Defect::fromFailedExecution` impide crear un defecto desde una ejecución que no sea `FAILED`. `registerRetest` exige versión corregida y aplica la transición a `CLOSED` o `REOPENED`. `Evidence` valida prefijo privado, checksum y metadatos, incluida la búsqueda recursiva de claves sensibles. Los enums eliminan estados arbitrarios dentro del dominio.

8.2 Aplicación

`ManageQaRegression` es el único caso de uso coordinador. Para crear una campaña verifica antes que cada caso exista en el hospital. Al registrar una ejecución comprueba membresía en campaña, convierte estado y severidad, construye evidencia y, cuando corresponde, crea el defecto enlazado. En una re-prueba recupera defecto y ejecución original, genera una ejecución nueva y delega la operación atómica. La aplicación recibe `hospitalId` y `userId` del contexto preparado por HTTP, no de campos confiados al cliente.

8.3 Infraestructura

El adaptador en memoria conserva colecciones útiles para pruebas. El adaptador Eloquent realiza consultas con hospital y usa transacciones para campaña, ejecución y re-prueba. En una re-prueba toma un bloqueo `lockForUpdate` sobre el defecto, vuelve a comprobar que no esté cerrado, inserta ejecución/evidencia y actualiza estado, versión y cierre. Este bloqueo reduce el riesgo de dos re-pruebas concurrentes incompatibles.

Los modelos Eloquent corresponden a seis tablas y convierten JSON y fechas según el contrato. `SpatieQaAuthorizer` integra el permiso de gestión. `RoleSeeder` crea `AnalistaQA` y asigna `qa.regression.manage`.

8.4 HTTP

Los requests validan forma, UUID, enums y campos condicionales antes del dominio. El controlador devuelve 201 para creaciones y re-pruebas, 200 para matriz, 422 para validación, 404 para recursos no visibles en el hospital, 409 para conflictos conocidos y 403 para autorización. JWT inválido o ausente queda bajo responsabilidad del middleware.

9. Modelo PostgreSQL

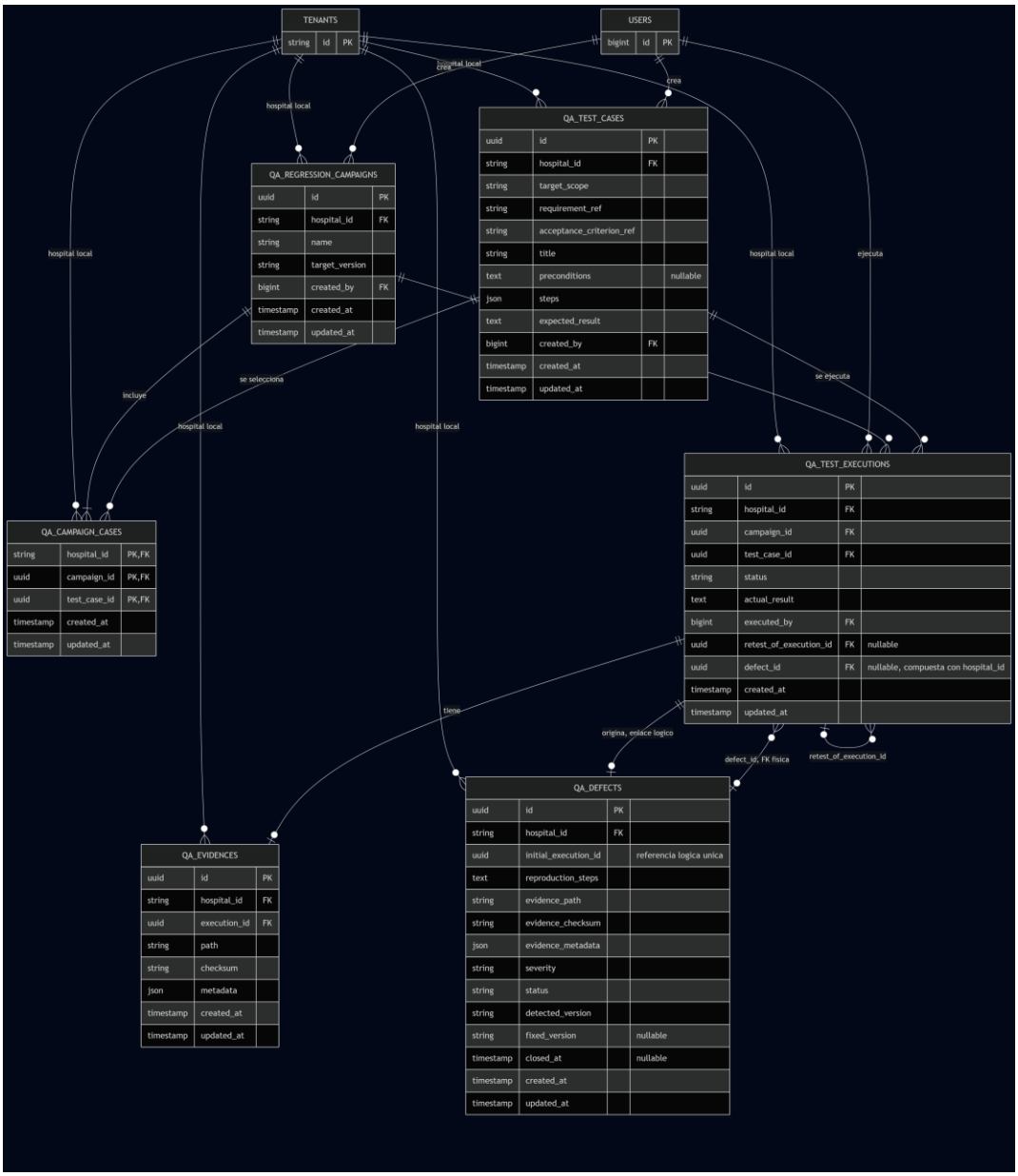
La migración `2026_08_21_000029_create_qa_regression_tables.php` crea `qa_test_cases`, `qa_regression_campaigns`, `qa_campaign_cases`, `qa_defects`, `qa_test_executions` y `qa_evidences`. La vista de datos muestra las relaciones físicas y distingue el enlace lógico `initial_execution_id` de la FK física `defect_id`.

Los casos y campañas usan UUID y conservan `hospital_id`. La tabla puente vincula hospital, campaña y caso mediante `PRIMARY KEY (hospital_id, campaign_id,`

test_case_id). Las ejecuciones referencian esa membresía mediante una FK compuesta, por lo que PostgreSQL puede rechazar una ejecución de un caso no asociado. qa_defects conserva la ejecución inicial como identificador lógico y duplica la referencia de evidencia inicial para mantener contexto. Las ejecuciones pueden referenciar un defecto y otra ejecución del mismo hospital. Las evidencias se vinculan a su ejecución mediante clave compuesta y un único (hospital_id, execution_id) refuerza la cardinalidad de una evidencia por ejecución.

Las columnas steps, metadata y evidence_metadata son JSON. La base no incorpora CHECK para enums o estructura interna; dichas reglas están en requests y dominio. Existen únicos para trazabilidad/título de caso, nombre/versión de campaña y asociaciones relevantes, además de índices para requisito y consulta de defectos. No se afirma uso de JSONB o índices GIN porque no están implementados.

La atomicidad depende de una única base PostgreSQL local. Crear campaña inserta cabecera y asociaciones en una transacción. Registrar una ejecución inserta ejecución y evidencia; si falla inicialmente, también crea el defecto en esa unidad. La re-prueba bloquea y actualiza el defecto dentro de la misma transacción. No hay coordinación distribuida con CENTRAL ni con un almacén físico de archivos.



Vista de datos PostgreSQL

Figura 4. Seis tablas QA, relaciones locales y claves de tenant. Fuente editable: `vista-datos.mmd`.

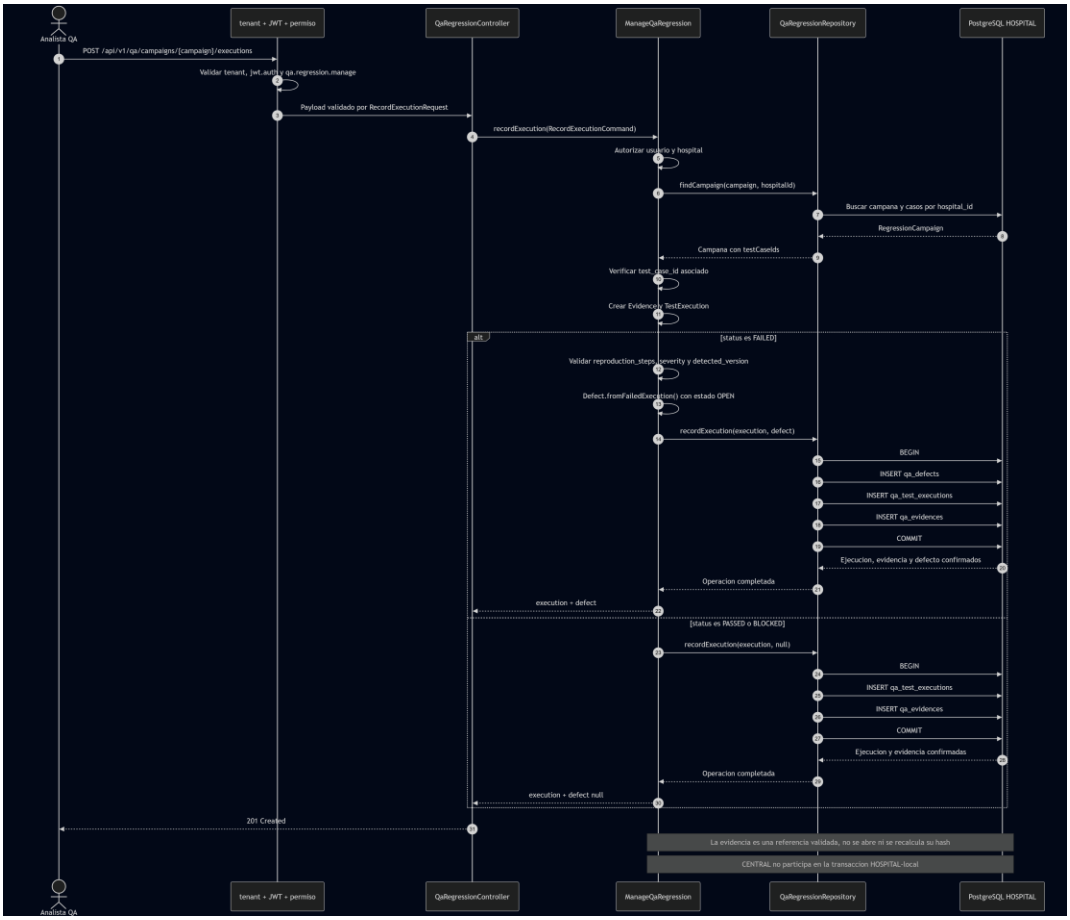
10. API entregada

La API publica exactamente cinco rutas:

Método	Ruta	Respuesta exitosa
POST	`/api/v1/qa/test-cases`	201
POST	`/api/v1/qa/campaigns`	201

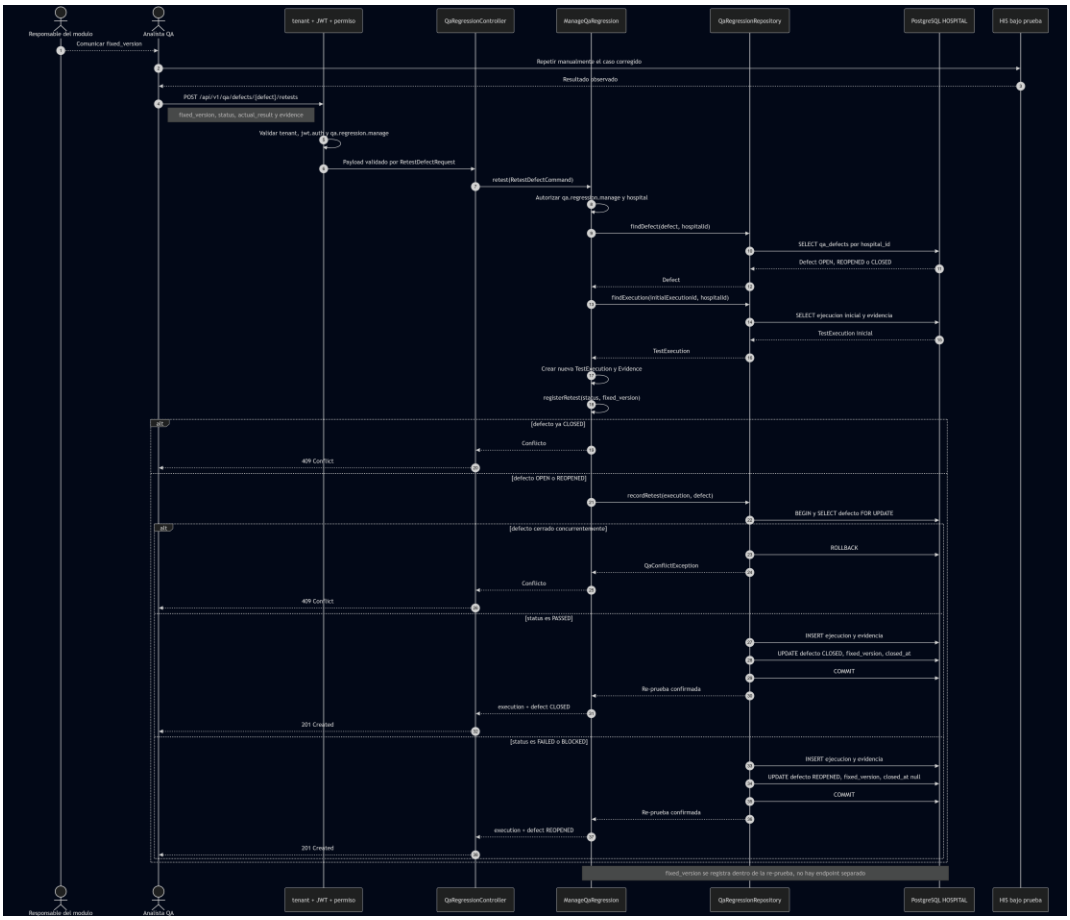
Método	Ruta	Respuesta exitosa
POST	`/api/v1/qa/campaigns/{campaign}/executions`	201
POST	`/api/v1/qa/defects/{defect}/retests`	201
GET	`/api/v1/qa/campaigns/{campaign}/matrix`	200

Todas aplican `jwt.auth`, `tenant` y `permission:qa.regression.manage,api`. El cliente envía su token y encabezado de tenant, pero no decide `hospital_id`, `created_by` ni `executed_by`. Los diagramas de secuencia muestran la validación, autorización, orden real de inserción, bloqueo del defecto y sus resultados alternos.



Secuencia de registro de ejecución

Figura 5. Ejecución inicial y creación atómica del defecto. Fuente editable: `secuencia-ejecucion.mmd`.



Secuencia de defecto y re-prueba

Figura 6. Re-prueba, concurrencia y cierre o reapertura. Fuente editable: `secuencia-defecto-reprueba.mmd`.

La matriz devuelve la campaña y una fila por caso con `latest_status`, ejecuciones con evidencia y defectos relacionados. Un caso aún no ejecutado muestra `latest_status=null`. La consulta realiza varias lecturas filtradas; no se presenta como una instantánea serializable de toda la campaña.

11. Seguridad y privacidad

La primera defensa es el contexto de acceso: JWT identifica al usuario, tenant delimita el hospital y Spatie verifica el permiso. Las pruebas confirman 401 para usuario eliminado, 403 sin permiso y 403 cuando un token intenta usar el contexto de otro tenant. En aplicación, un caso de otro hospital aparece como no encontrado, evitando exponer su existencia.

La evidencia se trata como una referencia declarada. La ruta privada evita representar el artefacto como URL pública; SHA-256 brinda un identificador declarado; `redacted=true`, MIME y tamaño obligan a describir saneamiento y forma. El rechazo recursivo de claves

sensibles reduce filtraciones de tokens, cookies o contraseñas anidadas. Las pruebas usan hospitales, usuarios y correos ficticios. No se utilizaron datos reales de pacientes ni credenciales institucionales.

Debe conservarse una limitación: el módulo no abre el archivo, no comprueba su existencia y no recalcula el checksum. Por tanto, la evidencia queda registrada, pero no verificada físicamente. Una política operativa debe exigir que el analista redacte nombres, identificadores clínicos y otros datos personales antes de referenciar el artefacto.

12. Pruebas y evidencia

El entorno verificado utilizó PHP 8.3.31, Laravel 12.58.0 y PHPUnit 11.5.55. PostgreSQL 17 se ejecutó mediante `postgres:17-alpine` en un contenedor aislado y la base destructible `shi_mod29_test`. La prueba exige sufijo `_test` y `QA_PG_ALLOW_MIGRATE_FRESH=1`, lo que reduce el riesgo de ejecutar `migrate:fresh` contra una base no destinada a QA.

La orden `php -d extension=pdo_pgsql vendor/bin/phpunit tests/Unit/Modulo29 tests/Feature/Modulo29` terminó en PASS con 21 pruebas y 63 aserciones. Cubre invariantes de dominio, aplicación con Repository en memoria, flujo HTTP, autorización, tenant, metadatos sensibles, re-prueba y persistencia PostgreSQL.

Nivel	Pruebas	Aserciones destacadas
Dominio	7	Referencias, estados, evidencia privada y metadatos sensibles
Aplicación con InMemory Repository	6	Flujo, aislamiento y ausencia de mutación ante re-prueba inválida
HTTP/API	6	Cinco rutas, JWT, tenant, permiso y validación 422
Integración PostgreSQL	2	13 aserciones sobre atomicidad, re-prueba, matriz, tenant y unicidad de evidencia
Total	**21**	**63 aserciones**

La regresión del Módulo 20 terminó en PASS con 13 pruebas y 30 aserciones. Además, `php -d extension=pdo_pgsql artisan migrate:fresh --seed --force`, dirigido a `shi_mod29_test`, completó las 12 migraciones, incluida la del Módulo 29, y todos los seeders. `git diff --check` pasó. Pint focal sobre archivos del módulo también pasó. Pint global reporta deuda de formato heredada fuera de Modulo29; ese hallazgo no representa un fallo de formato del módulo entregado y no se alteró código ajeno para ocultarlo.

Los resultados detallados, la matriz ejecutada y la transparencia del apoyo técnico se encuentran respectivamente en `EVIDENCIA.md`, `MATRIZ_REGRESION.md` y `DECLARACION_IA.md`.

13. Matriz, defecto y re-prueba

La matriz ejecutada documenta seis escenarios. Se verificó la creación de caso/campaña trazables, la creación atómica de defecto `OPEN`, el rechazo entre tenants, el rechazo de metadata sensible, la reapertura después de una re-prueba fallida y la compatibilidad con PostgreSQL. Los cinco primeros terminaron en PASS directamente mediante pruebas de dominio, aplicación o HTTP.

El sexto conserva una historia distinta. Durante la validación prepublicación, PostgreSQL rechazó la FK compuesta autorreferente de ejecuciones con `SQLSTATE 42830`: el orden de la referencia no coincidía con la clave única que debía respaldarla. Se registró `QA-DEF-001`, severidad `HIGH`, versión detectada `mod29-0.1.0-pre`. El impacto impedía completar la migración real, aunque no afectó datos reales porque se trabajó en un contenedor aislado.

La corrección se incluyó en `0e5aa7c`, alineando la estructura dentro de la migración del Módulo 29 y sin modificar otro módulo. La re-prueba en `80f8f37` terminó en PASS: una prueba PostgreSQL, seis aserciones, 12 migraciones y seeders exitosos. El registro se mantiene como `FAILED → FIXED → PASS`; no se reescribe la historia como si el defecto nunca hubiera existido.

14. Decisiones, riesgos y limitaciones

La decisión principal fue concentrar la orquestación en `ManageQaRegression` y colocar persistencia detrás de un puerto orientado a operaciones. Esto permite pruebas rápidas y un adaptador PostgreSQL sin acoplar dominio a Eloquent. La persistencia local por hospital evita que la indisponibilidad CENTRAL bloquee el registro de QA. Las transacciones y claves compuestas protegen consistencia interna.

Los riesgos aceptados incluyen ausencia de versionado o snapshots: la trazabilidad histórica depende del caso actual, aunque la API no ofrece edición. El archivo de evidencia no se verifica. Los estados y JSON no tienen `CHECK` en base, por lo que escrituras externas podrían romper invariantes. La matriz usa varias consultas y no ofrece una instantánea global. Tampoco existe idempotencia; repetir una solicitud tras un timeout puede generar otro registro salvo que una restricción única real lo impida.

No hay outbox ni sincronización federada. Incorporarlos sin necesidad habría añadido contratos de privacidad, reintentos y duplicados que exceden el MVP. Si se requiere consolidar métricas en CENTRAL, deberá diseñarse como otra capacidad. Del mismo modo, un almacén físico de evidencia necesitaría autorización, cifrado, retención y verificación de contenido antes de poder afirmar que el checksum fue comprobado.

15. Conclusiones

El Módulo 29 entrega un vertical mínimo pero completo para documentar regresión funcional. La trazabilidad comienza en requisito y criterio de aceptación, continúa por campaña y ejecución, conserva evidencia declarada y llega a defecto, versión corregida y re-prueba. Las cinco rutas están protegidas y los datos permanecen vinculados al hospital autenticado.

El patrón Repository demostró utilidad concreta: el mismo caso de uso fue validado con almacenamiento en memoria y con Eloquent/PostgreSQL, mientras las transacciones permanecieron como responsabilidad de infraestructura. La distinción frente a Git y a la federación evita atribuir al patrón funciones que no posee.

La validación sobre PostgreSQL 17 fue esencial. Detectó `QA-DEF-001`, invisible en una revisión puramente conceptual, y permitió corregirlo antes de publicación. La re-prueba, la regresión de Modulo29 y la regresión focal de Modulo20 ofrecen evidencia reproducible del estado `80f8f37`. El resultado no elimina las limitaciones declaradas, pero sí cumple el alcance sin afirmar versionado, snapshots, outbox, idempotencia o verificación física inexistentes.

16. Bibliografía

1. Fowler, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. Sección “Repository”.
2. Laravel. “Database: Getting Started” y “Database Transactions”. https://laravel.com/docs/12.x/database
3. Laravel. “HTTP Routing” y “Validation”. https://laravel.com/docs/12.x/routing y https://laravel.com/docs/12.x/validation
4. PostgreSQL Global Development Group. “Constraints”. PostgreSQL 17 Documentation. https://www.postgresql.org/docs/17/ddl-constraints.html
5. PHPUnit. “The Command-Line Test Runner”. PHPUnit 11.5 Manual. https://docs.phpunit.de/en/11.5/textui.html
6. Spatie. “Laravel Permission”. https://spatie.be/docs/laravel-permission
7. Stancl. “Tenancy for Laravel”. https://tenancyforlaravel.com/docs/v3/
8. Repositorio del proyecto, `docs/modulos/mod29/ESPECIFICACION.md`, `ADR-001-arquitectura.md`, fuentes Mermaid, código y pruebas en el corte `80f8f37`.